Statistical Arbitrage — Full Code Audit & Continuation Roadmap

> Based on your current codebase + TWS API live trading target

---

PART 1 — BUGS TO FIX RIGHT NOW

These will crash or silently corrupt results before you write a single new line.

BUG 1 — `ou_fitting.py`: variable used before assignment (CRASH)

File: `core/ou_fitting.py`, function `fit_hedge_ratio`, line ~65

Problem:

```python
# BROKEN — mu and sigma are checked BEFORE being computed
if mu <= 0 or sigma <= 0:
    continue

try:
    mu, theta, sigma = compute_ou_params(spread)   # ← assigned AFTER the check
```

The `if mu <= 0` guard runs before `mu` exists → `NameError` on first iteration.

Fix:

```python
for b in b_values:
    spread = s1 - b * s2
    try:
        mu, theta, sigma = compute_ou_params(spread)   # compute FIRST
        if mu <= 0 or sigma <= 0:                      # guard AFTER
            continue
        ll = compute_log_likelihood(spread, mu, theta, sigma)
        if ll > best_ll:
            best_ll     = ll
            best_b      = b
            best_params = (mu, theta, sigma)
    except Exception as e:
        continue   # silent skip on numerical failures
```

BUG 2 — `ou_fitting.py`: sigma² formula is wrong (SILENT WRONG RESULTS)

File: `core/ou_fitting.py`, function `compute_ou_params`, sigma calculation

Problem:

```python
# BROKEN — missing the cross-terms, formula doesn't match the paper (page 19)
var = ((2*mu)*Xyy - 2*(alpha**2)*Xxy + alpha**2 *Xxx
       - 2*theta*(1-alpha)*(Xy - alpha*Xx)
       + (n*theta**2)*(1-alpha)**2) / (n*(1 - alpha**2))
```

The paper's formula on page 19 is:

```
σ²* = (2μ* / n(1 − e^{−2μ*Δt})) × (Xyy − 2e^{−μ*Δt}Xxy + e^{−2μ*Δt}Xxx
       − 2θ*(1−e^{−μ*Δt})(Xy − e^{−μ*Δt}Xx) + n(θ*)²(1−e^{−μ*Δt})²)
```

Fix:

```python
alpha  = np.exp(-mu * dt)
alpha2 = np.exp(-2 * mu * dt)    # e^{-2μΔt} ← distinct from alpha²

numerator = (
    Xyy
    - 2 * alpha  * Xxy
    + alpha2     * Xxx
    - 2 * theta  * (1 - alpha) * (Xy - alpha * Xx)
    + n * theta**2 * (1 - alpha)**2
)
var   = (2 * mu / (n * (1 - alpha2))) * numerator
sigma = np.sqrt(max(var, 1e-12))   # clamp to avoid sqrt of negative from numerical
```

BUG 3 — `spread.py`: wrong pandas API (CRASH)

File: `core/spread.py`

Problem:

```python
# BROKEN — these methods don't exist on a Series/DataFrame
rl_mean = spread.rolling_mean(M)   # AttributeError
rl_std  = spread.rolling_std(M)    # AttributeError
```

Fix:

```python
def rolling_mean(spread, M):
    return spread.rolling(window=M).mean()

def rolling_std(spread, M):
    return spread.rolling(window=M).std(ddof=1)
```

BUG 4 — `spread.py` : `simulate_ou` is empty (MISSING IMPLEMENTATION)

File: `core/spread.py`

Problem:

```python
def simulate_ou(mu, theta, sigma, T, dt, n_paths):
    returnimport pandas as pd    # ← file concatenation artifact, crashes immediatel
```

Fix:

```python
def simulate_ou(mu, theta, sigma, T, dt, n_paths, x0=None):
    """
    Euler-Maruyama discretization of dX = mu*(theta-X)dt + sigma*dW
    Returns array of shape (n_paths, n_steps+1)
    """
    n_steps = int(T / dt)
    x0      = theta if x0 is None else x0
    paths   = np.zeros((n_paths, n_steps + 1))
    paths[:, 0] = x0

    for t in range(1, n_steps + 1):
        dW = np.random.normal(0, np.sqrt(dt), size=n_paths)
        paths[:, t] = (paths[:, t-1]
                       + mu * (theta - paths[:, t-1]) * dt
                       + sigma * dW)
    return paths
```

BUG 5 — `backtest/engine.py` : `_execute_signal` signature mismatch (CRASH)

File: `backtest/engine.py`

Problem:

```python
# run() calls with 3 args:
pnl = self._execute_signal(signal, price, self.capital)

# but _execute_signal is defined with 5 args:
def _execute_signal(self, signal, price, capital, tick, date):
#                                                ^^^^  ^^^^  ← never passed by run
```

Also: `run()` never appends to `self.pnl_log`, so `pd.DataFrame(self.pnl_log)` always returns an empty frame.

Fix — rewrite `run()` and align the signature:

```python
def run(self, pair_id, strategy, price_data: pd.DataFrame) -> pd.DataFrame:
    self.capital   = self.initial_capital
    self.positions = {}
    self.pnl_log   = []

    for date, row in price_data.iterrows():
        signal = strategy.generate_signal(date, price_data.loc[:date])
        price  = strategy.get_spread_price(date, price_data.loc[:date])
        pnl    = self._execute_signal(signal, price, self.capital,
                                      tick=pair_id, date=date)
        self.capital += pnl
        self.pnl_log.append({'date': date, 'pnl': pnl, 'capital': self.capital})

    return pd.DataFrame(self.pnl_log)
```

BUG 6 — `data/loader.py` : cache path has no directory prefix (SILENT WRONG CACHE)

File: `data/loader.py`

Problem:

```python
file = f"{tick}_data_{itv}"   # saves in whatever the CWD is
```

If you run from different directories you get cache misses and stale data silently.

Fix:

```python
CACHE_DIR = os.path.join(os.path.dirname(__file__), "..", "data", "cache")
os.makedirs(CACHE_DIR, exist_ok=True)

def fetch_pair_prices(tickers, start, end, itv="1d"):
    df = pd.DataFrame()
    for tick in tickers:
        file = os.path.join(CACHE_DIR, f"{tick}_{itv}_{start}_{end}.parquet")
        if os.path.exists(file):
            stock_data = pd.read_parquet(file)
        else:
            stock_data = yf.download(tick, start, end, interval=itv,
                                     progress=False, auto_adjust=True)
            if stock_data.empty:
                raise ValueError(f"No data for {tick}")
            if isinstance(stock_data.columns, pd.MultiIndex):
                stock_data.columns = stock_data.columns.droplevel(1)
            stock_data.to_parquet(file)
        df[f"{tick}_close"] = stock_data["Close"]
    df.dropna(inplace=True)
    return df
```

BUG 7 — `configs/default.yaml`: requirements.txt content is concatenated into it (FILE CORRUPTION)

File: `configs/default.yaml`

The file ends with the requirements.txt content pasted directly in:

```yaml
  n_train_paths: 10000yfinance   # ← requirements.txt starts here
pandas
numpy
...
```

This will crash any YAML parser.

Fix: Split into two clean files. The `default.yaml` should end at `n_train_paths: 10000`. Create a separate clean `requirements.txt`.

BUG 8 — `strategies/pairs_trading.py`: `OUPairTrader` is broken by design (INCOMPLETE)

File: `strategies/pairs_trading.py`

Problems:

- `__init__` takes a `config` dict but never stores any attributes from it
- `fit()` references `self.start`, `self.end`, `self.interval`, `self.pair` — none of which are set in `__init__`
- The function body is cut off mid-loop (`for ` with nothing after it)
- Import is `from data import loader` — will fail unless run from the project root; should be a relative import

This file needs a full rewrite (see Phase 2 steps below).

BUG 9 — `core/metrics.py`: `performance_summary` returns wrong types (LOGIC ERROR)

File: `core/metrics.py`

```python
# BROKEN — returns full Series objects instead of scalar summaries
"daily_sharpe": daily_sharpe(pnl_series, rf=rf, ann=ann),   # expanding Series
"annual_pnl":   annual_pnl(pnl_series),                     # Series by year
"annual_max_dd": annual_max_drawdown(pnl_series),           # Series by year
```

For a backtest report you want scalar values, not time series inside a dict.

Fix:

```python
def performance_summary(pnl_series, rf=0.0, ann=252):
    dr = daily_return(pnl_series)
    return {
        "avg_daily_return_pct": round(dr.mean() * 100, 4),
        "daily_std_pct":        round(dr.std(ddof=1) * 100, 4),
        "sharpe":               round(np.sqrt(ann) * dr.mean() / dr.std(ddof=1), 4),
        "max_drawdown_pct":     round(annual_max_drawdown(pnl_series).min() * 100, 4
        "total_pnl_pct":        round(annual_pnl(pnl_series).sum() * 100, 4),
    }
```

PART 2 — WHAT YOU HAVE VS WHAT YOU NEED

File	Status	Notes
`configs/default.yaml`	⚠ Corrupted	Fix file split (Bug 7)
`requirements.txt`	⚠ Corrupted	Separate from yaml
`data/loader.py`	✅ ~80% done	Fix cache path (Bug 6)
`core/ou_fitting.py`	⚠ 2 bugs	Fix bugs 1 & 2 — critical
`core/spread.py`	❌ Broken	Fix bugs 3 & 4
`core/metrics.py`	✅ ~70% done	Fix Bug 9
`backtest/engine.py`	⚠ Partially broken	Fix Bug 5, add pnl_log
`backtest/report.py`	❌ Empty	Needs full implementation
`strategies/base.py`	❌ Empty	Needs interface definition
`strategies/pairs_trading.py`	❌ Broken	Needs full rewrite
`strategies/diversification.py`	❌ Empty	Not started
`strategies/signature_sot.py`	❌ Empty	Not started
`strategies/rl_trading.py`	❌ Empty	Not started
`tests/`	❌ All empty	Not started
TWS API layer	❌ Missing	Not created yet

PART 3 — STEP-BY-STEP CONTINUATION PLAN

STEP 1 — Fix all 9 bugs above (1–2 hours)

Apply every fix from Part 1 in order. After fixing:

```bash
```

```
# Quick smoke test — should print (mu, theta, sigma) without crashing
python -c "
from core.ou_fitting import compute_ou_params, fit_hedge_ratio
import numpy as np
np.random.seed(42)
# Simulate OU with known params: mu=10, theta=5, sigma=1
dt=1/252; T=252; n=int(T/dt)
x = np.zeros(n); x[0]=5
for i in range(1,n):
    x[i] = x[i-1] + 10*(5-x[i-1])*dt + 1*np.sqrt(dt)*np.random.randn()
mu, theta, sigma = compute_ou_params(x, dt=dt)
print(f'mu={mu:.2f} (expect ~10), theta={theta:.2f} (expect ~5), sigma={sigma:.2f} (
"
```

STEP 2 — Fix `requirements.txt` and install (15 minutes)

Create a clean `requirements.txt`:

```
yfinance>=0.2.40
pandas>=2.0
numpy>=1.26
scipy>=1.12
scikit-learn>=1.4
matplotlib>=3.8
seaborn>=0.13
iisignature>=0.24
torch>=2.2
pyyaml>=6.0
pytest>=8.0
jupyter>=1.0
tqdm>=4.66
ibapi>=10.19.2          # TWS API — install manually from IBKR site
```

Note: `ibapi` is NOT on PyPI. Download it from Interactive Brokers' website and install with:

```bash
cd ~/Downloads/TWS_API/source/pythonclient
python setup.py install
```

STEP 3 — Add `core/emrt.py` (Chapter 4 spread construction)

Create this new file. It is completely missing and needed for Chapter 4:

```python
```

```python
# core/emrt.py
import numpy as np

def find_important_extrema(series: np.ndarray, C: float = 2.0):
    """
    Returns list of (index, 'min'|'max') for all important extrema.
    A point is an important min if both surrounding endpoints are >= C*std above it.
    """
    s   = np.array(series)
    std = s.std()
    extrema = []
    n = len(s)

    for m in range(1, n - 1):
        # Check important minimum
        left_max  = s[:m].max()  if m > 0      else s[0]
        right_max = s[m+1:].max() if m < n-1   else s[-1]
        if (left_max - s[m] >= C * std) and (right_max - s[m] >= C * std):
            if s[m] == s[max(0,m-5):m+6].min():   # local minimum check
                extrema.append((m, 'min'))
                continue

        # Check important maximum
        left_min  = s[:m].min()  if m > 0     else s[0]
        right_min = s[m+1:].min() if m < n-1  else s[-1]
        if (s[m] - left_min >= C * std) and (s[m] - right_min >= C * std):
            if s[m] == s[max(0,m-5):m+6].max():   # local maximum check
                extrema.append((m, 'max'))

    return extrema


def compute_emrt(series: np.ndarray, C: float = 2.0) -> float:
    """
    Empirical Mean Reversion Time:
    Average time from a local extreme back to the sample mean.
    """
    s     = np.array(series)
    mean  = s.mean()
    extr  = find_important_extrema(s, C)
    if len(extr) < 2:
        return np.inf

    durations = []
    for idx, kind in extr:
        # Find first crossing of mean after this extreme
```

```
        future = s[idx:]
        if kind == 'min':
            crossing = np.where(future >= mean)[0]
        else:
            crossing = np.where(future <= mean)[0]
        if len(crossing) > 0:
            durations.append(crossing[0])

    return float(np.mean(durations)) if durations else np.inf


def fit_hedge_ratio_emrt(s1, s2, grid_min=-3.0, grid_max=3.0,
                         grid_step=0.01, C=2.0, max_var=None):
    """
    Find B minimizing EMRT of (s1 - B*s2).
    max_var: optional upper bound on spread variance (leverage control).
    """
    s1 = np.array(s1)
    s2 = np.array(s2)
    b_values  = np.arange(grid_min, grid_max + grid_step, grid_step)
    best_b    = 1.0
    best_emrt = np.inf

    for b in b_values:
        spread = s1 - b * s2
        if max_var is not None and spread.var() > max_var:
            continue
        emrt = compute_emrt(spread, C)
        if emrt < best_emrt:
            best_emrt = emrt
            best_b    = b

    return best_b, best_emrt
```

STEP 4 — Rewrite `strategies/base.py`

Every strategy must implement this interface so the backtest engine can call them uniformly:

python

```python
# strategies/base.py
from abc import ABC, abstractmethod
import pandas as pd

class BaseStrategy(ABC):

    @abstractmethod
    def fit(self, formation_data: pd.DataFrame) -> None:
        """Calibrate model on formation-period data."""

    @abstractmethod
    def generate_signal(self, date, history: pd.DataFrame) -> int:
        """Return +1 (long spread), -1 (short spread), 0 (flat)."""

    def get_spread_price(self, date, history: pd.DataFrame) -> float:
        """Return the spread value at `date`. Override if needed."""
        return float(history["spread"].iloc[-1])
```

STEP 5 — Rewrite `strategies/pairs_trading.py`

Full clean rewrite:

```python
```

```python
# strategies/pairs_trading.py
import numpy as np
import pandas as pd
from strategies.base import BaseStrategy
from core.ou_fitting import fit_hedge_ratio, compute_ou_params
from core.spread     import construct_spread, rolling_mean, rolling_std

class OUPairTrader(BaseStrategy):

    def __init__(self, config: dict):
        t          = config["trading"]
        self.K     = t.get("K",            1.0)
        self.M     = t.get("M",            63)
        self.r     = t.get("stop_loss_r",  0.1)
        ou         = config.get("ou", {})
        grid       = ou.get("hedge_ratio_grid", [-2.0, 2.0, 0.01])
        self.g_min, self.g_max, self.g_step = grid

        # Set after fit()
        self.B     = None
        self.mu    = None
        self.theta = None
        self.sigma = None

        # Runtime state
        self._position   = 0        # +1 long, -1 short, 0 flat
        self._entry_val  = None     # spread value at entry
        self._s1_col     = None
        self._s2_col     = None

    def fit(self, formation_data: pd.DataFrame) -> None:
        cols = formation_data.columns.tolist()
        self._s1_col, self._s2_col = cols[0], cols[1]
        s1 = formation_data[self._s1_col].values
        s2 = formation_data[self._s2_col].values
        self.B, self.mu, self.theta, self.sigma = fit_hedge_ratio(
            s1, s2, self.g_min, self.g_max, self.g_step
        )

    def generate_signal(self, date, history: pd.DataFrame) -> int:
        if self.B is None:
            raise RuntimeError("Call fit() before generate_signal()")

        spread = construct_spread(history[self._s1_col], history[self._s2_col], self
        ma     = rolling_mean(spread, self.M)
        sd     = rolling_std(spread,  self.M)
```

```
        current = spread.iloc[-1]
        ma_val  = ma.iloc[-1]
        sd_val  = sd.iloc[-1]

        if np.isnan(ma_val) or np.isnan(sd_val) or sd_val == 0:
            return 0

        # Stop-loss
        if self._position != 0 and self._entry_val is not None:
            if self._position == 1 and current < self._entry_val * (1 - self.r):
                self._position = 0;  return 0
            if self._position == -1 and current > self._entry_val * (1 + self.r):
                self._position = 0;  return 0

        # Exit
        if self._position == 1  and current > ma_val:
            self._position = 0;  return 0
        if self._position == -1 and current < ma_val:
            self._position = 0;  return 0

        # Entry
        if self._position == 0:
            if current < ma_val - self.K * sd_val:
                self._position = 1;  self._entry_val = current;  return 1
            if current > ma_val + self.K * sd_val:
                self._position = -1; self._entry_val = current;  return -1

        return self._position   # hold

    def get_spread_price(self, date, history: pd.DataFrame) -> float:
        spread = construct_spread(history[self._s1_col], history[self._s2_col], self
        return float(spread.iloc[-1])
```

STEP 6 — Implement `strategies/diversification.py`

python

```python
# strategies/diversification.py
import numpy as np
from scipy.optimize import minimize

def _minmax(arr):
    lo, hi = arr.min(), arr.max()
    return (arr - lo) / (hi - lo + 1e-12)

class EqualWeightAllocator:
    def compute_weights(self, n):
        return np.ones(n) / n

class MRBAllocator:
    def compute_weights(self, mus, sigmas, likelihoods):
        mus  = np.array(mus,         dtype=float)
        sigs = np.array(sigmas,      dtype=float)
        lls  = np.array(likelihoods,dtype=float)
        mu_r      = mus / (sigs + 1e-12)
        mu_tilde  = _minmax(mu_r)
        ll_tilde  = _minmax(lls)
        raw       = mu_tilde * ll_tilde
        total     = raw.sum()
        if total == 0:
            return np.ones(len(mus)) / len(mus)
        return raw / total

class MRRAllocator:
    def compute_weights(self, mus, sigmas, likelihoods):
        n    = len(mus)
        mus  = np.array(mus,         dtype=float)
        sigs = np.array(sigmas,      dtype=float)
        lls  = np.array(likelihoods,dtype=float)
        mu_r     = mus / (sigs + 1e-12)
        mu_tilde = _minmax(mu_r)
        ll_tilde = _minmax(lls)
        scores   = mu_tilde * ll_tilde
        order    = np.argsort(scores)          # ascending
        weights  = np.zeros(n)
        for rank, idx in enumerate(order):
            # Formula (2.4) from paper
            weights[idx] = (n - 1 + 2 * rank) / (2 * n * (n - 1) + 1e-12)
        weights /= weights.sum()
        return weights

class MVAAllocator:
    def compute_weights(self, returns_matrix):
```

```python
    """returns_matrix: shape (T, N) daily returns per pair"""
    R   = np.array(returns_matrix)
    mu  = R.mean(axis=0)
    cov = np.cov(R.T)
    n   = len(mu)

    def neg_sharpe(w):
        w    = np.abs(w); w /= w.sum()
        ret  = w @ mu
        vol  = np.sqrt(w @ cov @ w + 1e-12)
        return -ret / vol

    result = minimize(neg_sharpe, np.ones(n)/n,
                      method='SLSQP',
                      constraints={'type':'eq','fun': lambda w: w.sum()-1},
                      bounds=[(0, 1)] * n)
    w = np.abs(result.x); return w / w.sum()
```

STEP 7 — Write unit tests (validates everything above)

```python
```

```python
# tests/test_ou_fitting.py
import numpy as np, pytest
from core.ou_fitting import compute_ou_params, fit_hedge_ratio

def simulate_ou(mu, theta, sigma, n=2000, dt=1/252, seed=42):
    rng = np.random.default_rng(seed)
    x = np.zeros(n); x[0] = theta
    for i in range(1, n):
        x[i] = x[i-1] + mu*(theta-x[i-1])*dt + sigma*np.sqrt(dt)*rng.standard_normal
    return x

def test_ou_params_recovery():
    x = simulate_ou(mu=15, theta=5, sigma=1)
    mu_h, th_h, sig_h = compute_ou_params(x, dt=1/252)
    assert abs(mu_h  - 15) / 15  < 0.25, f"mu off: {mu_h}"
    assert abs(th_h  -  5) /  5  < 0.10, f"theta off: {th_h}"
    assert abs(sig_h -  1) /  1  < 0.20, f"sigma off: {sig_h}"

def test_fit_hedge_ratio_returns_valid():
    rng = np.random.default_rng(0)
    s2 = simulate_ou(mu=10, theta=5, sigma=0.5)
    s1 = 0.98 * s2 + rng.normal(0, 0.02, len(s2))
    B, mu, theta, sigma = fit_hedge_ratio(s1, s2, -2, 2, 0.05)
    assert B is not None
    assert mu > 0
    assert sigma > 0

# tests/test_spread.py
import numpy as np, pandas as pd
from core.spread import construct_spread, rolling_mean, rolling_std, simulate_ou

def test_construct_spread():
    s1 = pd.Series([10, 11, 12])
    s2 = pd.Series([5,  5,  5])
    assert list(construct_spread(s1, s2, 2.0)) == [0, 1, 2]

def test_rolling_api():
    s = pd.Series(range(100), dtype=float)
    assert rolling_mean(s, 10).iloc[-1] == pytest.approx(94.5)
    assert rolling_std(s, 10).iloc[-1]  > 0

def test_simulate_ou_shape():
    paths = simulate_ou(mu=10, theta=5, sigma=1, T=1, dt=1/252, n_paths=50)
    assert paths.shape == (50, 253)
```

Run with: `pytest tests/ -v`

STEP 8 — Add `backtest/report.py`

python

```python
# backtest/report.py
import pandas as pd
import matplotlib.pyplot as plt
from core.metrics import performance_summary

class BacktestReport:
    def __init__(self, results: dict):
        # results = {"EW": pnl_df, "MRB": pnl_df, ...}
        self.results = results

    def summary_table(self) -> pd.DataFrame:
        rows = {}
        for name, pnl_df in self.results.items():
            rows[name] = performance_summary(pnl_df)
        return pd.DataFrame(rows).T

    def plot_cumulative_returns(self, figsize=(12, 5)):
        fig, ax = plt.subplots(figsize=figsize)
        for name, pnl_df in self.results.items():
            daily = pnl_df.groupby("date")["pnl"].sum().cumsum()
            ax.plot(daily.index, daily.values, label=name)
        ax.set_title("Cumulative PnL")
        ax.set_xlabel("Date"); ax.set_ylabel("Cumulative PnL")
        ax.legend(); ax.grid(True, alpha=0.3)
        plt.tight_layout()
        return fig

    def plot_drawdown(self, figsize=(12, 4)):
        fig, ax = plt.subplots(figsize=figsize)
        for name, pnl_df in self.results.items():
            daily  = pnl_df.groupby("date")["pnl"].sum().cumsum()
            dd     = daily - daily.cummax()
            ax.fill_between(dd.index, dd.values, 0, alpha=0.3, label=name)
        ax.set_title("Drawdown"); ax.legend(); ax.grid(True, alpha=0.3)
        plt.tight_layout()
        return fig
```

STEP 9 — Create the TWS API layer (NEW MODULE)

Create a new folder `broker/` with these files:

```
broker/
├── __init__.py
├── tws_client.py       # EClient/EWrapper subclass
├── order_manager.py    # translate signals → IB orders
└── live_trader.py      # orchestrates everything in real time
```

`broker/tws_client.py` — skeleton:

python

```python
from ibapi.client  import EClient
from ibapi.wrapper import EWrapper
from ibapi.contract import Contract
from ibapi.order    import Order
import threading, time, queue

class TWSClient(EWrapper, EClient):

    def __init__(self, host="127.0.0.1", port=7497, client_id=1):
        EClient.__init__(self, self)
        self.host      = host
        self.port      = port
        self.client_id = client_id
        self._prices   = {}           # ticker → last price
        self._req_id   = 0
        self._ready    = threading.Event()

    # ── Connection ──────────────────────────────────────────
    def connect_and_run(self):
        self.connect(self.host, self.port, self.client_id)
        t = threading.Thread(target=self.run, daemon=True)
        t.start()
        self._ready.wait(timeout=10)

    def nextValidId(self, orderId):
        self._next_order_id = orderId
        self._ready.set()

    # ── Market data ─────────────────────────────────────────
    def request_price(self, ticker: str) -> int:
        self._req_id += 1
        contract = self._make_stock_contract(ticker)
        self.reqMktData(self._req_id, contract, "", False, False, [])
        return self._req_id

    def tickPrice(self, reqId, tickType, price, attrib):
        # tickType 4 = last price
        if tickType == 4 and price > 0:
            self._prices[reqId] = price

    def get_last_price(self, req_id: int, timeout=5) -> float:
        deadline = time.time() + timeout
        while time.time() < deadline:
            if req_id in self._prices:
                return self._prices[req_id]
            time.sleep(0.05)
```

```
        raise TimeoutError(f"No price received for req_id={req_id}")

    # ── Orders ──────────────────────────────────────────────────
    def place_market_order(self, ticker: str, qty: float, side: str):
        contract = self._make_stock_contract(ticker)
        order          = Order()
        order.action   = side.upper()   # "BUY" or "SELL"
        order.orderType = "MKT"
        order.totalQuantity = abs(qty)
        oid = self._next_order_id
        self._next_order_id += 1
        self.placeOrder(oid, contract, order)
        return oid

    def orderStatus(self, orderId, status, filled, remaining,
                    avgFillPrice, *args):
        print(f"Order {orderId}: {status}, filled={filled} @ {avgFillPrice}")

    def error(self, reqId, errorCode, errorString, *args):
        if errorCode not in (2104, 2106, 2158):   # suppress info msgs
            print(f"TWS Error {errorCode}: {errorString}")

    # ── Helpers ─────────────────────────────────────────────────
    @staticmethod
    def _make_stock_contract(ticker: str) -> Contract:
        c          = Contract()
        c.symbol   = ticker
        c.secType  = "STK"
        c.exchange = "SMART"
        c.currency = "USD"
        return c
```

`broker/order_manager.py` — translate spread signal to two legs:

python

```python
class OrderManager:
    """
    Convert a spread signal (+1/-1/0) into actual stock orders.
    Spread = S1 - B*S2:
      Long  spread (+1) → BUY  S1, SELL S2
      Short spread (-1) → SELL S1, BUY  S2
    """
    def __init__(self, tws_client, capital_per_pair: float):
        self.tws     = tws_client
        self.capital = capital_per_pair
        self._pos    = 0

    def execute(self, signal: int, s1_ticker: str, s2_ticker: str,
                p1: float, p2: float, B: float):

        if signal == self._pos:
            return   # already in the right state

        # Close existing position first
        if self._pos != 0:
            self._close(s1_ticker, s2_ticker, p1, p2, B)

        # Open new position
        if signal != 0:
            qty1 = self.capital / p1
            qty2 = (self.capital * B) / p2
            if signal == 1:   # long spread
                self.tws.place_market_order(s1_ticker, qty1, "BUY")
                self.tws.place_market_order(s2_ticker, qty2, "SELL")
            else:             # short spread
                self.tws.place_market_order(s1_ticker, qty1, "SELL")
                self.tws.place_market_order(s2_ticker, qty2, "BUY")

        self._pos = signal

    def _close(self, s1, s2, p1, p2, B):
        qty1 = self.capital / p1
        qty2 = (self.capital * B) / p2
        if self._pos == 1:
            self.tws.place_market_order(s1, qty1, "SELL")
            self.tws.place_market_order(s2, qty2, "BUY")
        else:
            self.tws.place_market_order(s1, qty1, "BUY")
            self.tws.place_market_order(s2, qty2, "SELL")
```

`broker/live_trader.py` — the main loop:

python

```python
import time, yaml
import pandas as pd
from data.loader       import fetch_pair_prices
from core.ou_fitting   import fit_hedge_ratio
from strategies.pairs_trading import OUPairTrader
from broker.tws_client  import TWSClient
from broker.order_manager import OrderManager

class LiveTrader:
    """
    Connects to TWS, fits the model on formation data, then runs
    a live signal loop checking every `poll_seconds`.
    """
    def __init__(self, config_path: str, capital_per_pair=10000):
        with open(config_path) as f:
            self.cfg = yaml.safe_load(f)
        self.capital = capital_per_pair
        self.tws     = TWSClient()
        self.traders = {}        # pair_key → OUPairTrader
        self.managers= {}        # pair_key → OrderManager

    def setup(self):
        """Fit all pairs on formation data."""
        d   = self.cfg["data"]
        for t1, t2 in d["tickers_ch2"]:
            data = fetch_pair_prices([t1, t2],
                                     d["formation_start"],
                                     d["formation_end"], "1d")
            trader = OUPairTrader(self.cfg)
            trader.fit(data)
            self.traders[(t1, t2)]  = trader
            print(f"Fitted {t1}-{t2}: B={trader.B:.3f}, mu={trader.mu:.2f}")

    def connect(self):
        self.tws.connect_and_run()
        print("Connected to TWS")

    def run(self, poll_seconds=60):
        """Main live loop — runs forever, checks every poll_seconds."""
        d = self.cfg["data"]
        while True:
            for (t1, t2), trader in self.traders.items():
                try:
                    # Fetch recent bar
                    history = fetch_pair_prices([t1, t2],
                                                d["trading_start"],
```

```
                                        pd.Timestamp.today().strftime("%Y-%m
                                        "1d")
                signal = trader.generate_signal(history.index[-1], history)
                p1 = history[f"{t1}_close"].iloc[-1]
                p2 = history[f"{t2}_close"].iloc[-1]

                mgr = self.managers.setdefault(
                    (t1, t2),
                    OrderManager(self.tws, self.capital)
                )
                mgr.execute(signal, t1, t2, p1, p2, trader.B)
                print(f"{t1}-{t2} signal={signal}")

            except Exception as e:
                print(f"Error on {t1}-{t2}: {e}")

        time.sleep(poll_seconds)
```

STEP 10 — Add a `run.py` entry point

python

```python
# run.py  (at project root)
import argparse, yaml
from backtest.engine   import BacktestEngine
from backtest.report   import BacktestReport
from data.loader       import fetch_pair_prices
from strategies.pairs_trading import OUPairTrader

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--mode",   choices=["backtest","live"], default="backtest")
    parser.add_argument("--config", default="configs/default.yaml")
    args = parser.parse_args()

    with open(args.config) as f:
        cfg = yaml.safe_load(f)

    if args.mode == "backtest":
        run_backtest(cfg)
    else:
        from broker.live_trader import LiveTrader
        lt = LiveTrader(args.config)
        lt.setup(); lt.connect(); lt.run()

def run_backtest(cfg):
    d       = cfg["data"]
    results = {}
    for t1, t2 in d["tickers_ch2"]:
        form = fetch_pair_prices([t1, t2], d["formation_start"], d["formation_end"],
        trad = fetch_pair_prices([t1, t2], d["trading_start"],   d["trading_end"],
        trader = OUPairTrader(cfg)
        trader.fit(form)
        engine = BacktestEngine(initial_capital=10000, config=cfg)
        pnl_df = engine.run(f"{t1}-{t2}", trader, trad)
        results[f"{t1}-{t2}"] = pnl_df

    report = BacktestReport(results)
    print(report.summary_table())
    report.plot_cumulative_returns().savefig("outputs/cumulative_returns.png")

if __name__ == "__main__":
    main()
```

PART 4 — ORDERED CHECKLIST

```
PHASE A — Fix existing code (do this first, nothing else works until it's done)
  [ ] Bug 1: fix variable-before-assignment in fit_hedge_ratio
  [ ] Bug 2: fix sigma² formula in compute_ou_params
  [ ] Bug 3: fix rolling_mean/rolling_std API in spread.py
  [ ] Bug 4: implement simulate_ou in spread.py
  [ ] Bug 5: fix _execute_signal signature + pnl_log in engine.py
  [ ] Bug 6: fix cache path in loader.py
  [ ] Bug 7: split default.yaml and requirements.txt
  [ ] Bug 8: rewrite pairs_trading.py (Step 5 above)
  [ ] Bug 9: fix performance_summary return types

PHASE B — Complete core modules
  [ ] Step 3: create core/emrt.py
  [ ] Step 4: write strategies/base.py interface
  [ ] Step 6: implement strategies/diversification.py
  [ ] Step 7: write and pass all unit tests
  [ ] Step 8: implement backtest/report.py

PHASE C — Validate Chapter 2
  [ ] Run single-pair backtest on all 6 pairs (2022)
  [ ] Confirm results roughly match paper Table 2.4
  [ ] Run diversification framework, confirm MRB > MRR > EW > MVA

PHASE D — Chapter 3 (Signature)
  [ ] Install and test iisignature library
  [ ] Implement strategies/signature_sot.py
  [ ] Validate on simulated OU data first
  [ ] Run on 4 real pairs (WM-RSG, UAL-DAL, V-MA, GS-MS)

PHASE E — Chapter 4 (RL)
  [ ] Validate core/emrt.py against Table 4.1
  [ ] Implement strategies/rl_trading.py
  [ ] Train on 10,000 simulated OU paths
  [ ] Run on 10 S&P500 pairs (2023 trading period)

PHASE F — Live trading
  [ ] Install ibapi from IBKR website
  [ ] Test TWSClient connection on Paper Trading account (port 7497)
  [ ] Test OrderManager with 1 pair, tiny size, Paper account
  [ ] Add position reconciliation (query existing positions at startup)
  [ ] Add kill switch (close all positions on SIGINT)
  [ ] Run live on Paper account for 2 weeks before real money
```

PART 5 — TWS API SETUP CHECKLIST

Before writing any live trading code, make sure these are in place:

1. **TWS or IB Gateway** installed and running
2. **Paper Trading account** enabled in Client Portal
3. In TWS: `Edit → Global Configuration → API → Settings`
   - Enable ActiveX and Socket Clients ✓
   - Socket port: **7497** (paper) or **7496** (live)
   - Allow connections from localhost ✓
4. Connect with `client_id=1` — each script must use a unique client_id
5. **Always test on Paper account first** — at minimum 2 weeks
6. Add position size limits in `OrderManager` before going live